PREVIOUS: VECTOR RAG STRATEGY

EXTENSION: GRAPH RAG LAYER

Adding a *Kn*
to Your RAG

This document extends th
layer — enabling relations
conflict detection, and en

Base: Existing Vector RAG +

Added: Dual Retrieval Router

This is a layered addition — nothing from Phase 1–10 is removed

Qdrant (vector search), BM25, PostgreSQL (parent chunk store), Cross-Encoder Reranker — all remain exactly as designed. The graph is a parallel retrieval path that activates when queries involve relationships, comparisons, dependencies, or entity-level reasoning. Both paths feed into the same reranker and LLM.

// CONTENTS — GRAPH RAG EXTENSION

G1 Why Graph + What It Solves — new
G2 Graph Schema Design (Neo4j) — new
G3 Entity Extraction Pipeline — new
G4 Graph Population Strategy — new
G5 Query Router — Vector vs Graph vs Both — new
G6 Cypher Query Patterns (Banking) — new
G7 GraphQL vs Cypher — Clarified — new
G8 Result Fusion from Both Paths — new
G9 Full Updated Architecture — new
G10 Updated Implementation Roadmap — new

G1 // GRAPH EXTENSION

Why Graph — What It Solves That Vector Cannot

Vector search answers "what does this text say about X?" — Graph answers "how does X relate to Y, Z, and W across all documents?"

> Pure vector search treats every chunk as an island. It retrieves based on semantic similarity but knows

nothing about how concepts *connect* to each other. The moment a user asks a relationship, comparison, or dependency question, vector search can only hope that both pieces of information landed in the same chunk — which they rarely do across 30+ documents.

Query Types That Break Pure Vector Search

✕ Vector RAG Fails On These

- **Comparison:** "How does ACH payment processing differ between FSD v1 and FSD v2?"
- **Dependency:** "What systems does the Genesys integration depend on, and what fails if RiskEngine is down?"
- **Conflict:** "Are there any conflicting rules about transaction limits across documents?"
- **Chain:** "What is the complete flow from customer onboarding to first successful remittance?"
- **Impact:** "If the OTP service changes, which workflows across all docs are affected?"
- **Aggregation:** "Which payment modes appear in more than 3 documents?"

✓ Graph RAG Handles These Natively

- **Traverse:** Follow edges between entities across documents and versions
- **Compare:** Find same entity in multiple docs and surface property differences
- **Dependency chain:** Walk dependency edges to find cascading effects
- **Conflict detect:** Find two nodes with same label but contradicting properties
- **Impact analysis:** Reverse-traverse "used_by" edges from any entity
- **Aggregation:** Count, filter, group by entity type natively in Cypher

The Core Mental Model

```
// Every entity in your 30 documents becomes a node in Neo4j
// Every mention, dependency, or rule becomes a typed edge

(:PaymentMode {name: "ACH", currency: "USD"})
    —[:USED_IN {section: "1.1", mandatory: true}]—► (:Document {id: "ICICI_ORP_FSD"})
    —[:CONFLICTS_WITH {reason: "limit_mismatch"}]—► (:PaymentMode {name: "ACH", limit: 50000}) ← from Doc_07

(:System {name: "Genesys"})
    —[:DEPENDS_ON]—► (:System {name: "RiskEngine"})
    —[:DEPENDS_ON]—► (:System {name: "OTPService"})
    —[:INTEGRATES_WITH]—► (:System {name: "CoreBanking"})

(:Workflow {name: "CustomerOnboarding"})
    —[:NEXT_STEP]—► (:Workflow {name: "KYCVerification"})
    —[:NEXT_STEP]—► (:Workflow {name: "RiskRatingAssignment"})
    —[:NEXT_STEP]—► (:Workflow {name: "FirstTransactionEnable"})

(:Rule {name: "TransactionLimit", value: "5000", currency: "USD"})
```

```
    —[:APPLIES_TO]——► (:CustomerSegment {type: "RetailNRI"})
    —[:DEFINED_IN]——► (:Document {id: "ICICI_ORP_FSD", section: "2.3"})
```

G2 // GRAPH SCHEMA

Neo4j Graph Schema — Designed for Banking FSDs

The schema defines what nodes exist, what properties they carry, and what relationships connect them. This is the backbone of everything.

Node Types (Labels)

NODE LABEL	WHAT IT REPRESENTS	KEY PROPERTIES	EXAMPLE
:Document	Each uploaded FSD/BRD file	id, name, type, version, date, system	ICICI_ORP_Genesys_FSD_v1.2
:Section	Every heading/section in a doc	doc_id, path, heading_text, level, chunk_id	"View Integration > Status of Funds"
:System	Software system / platform / service	name, type, vendor, environment	Genesys, CoreBanking, OTPService
:PaymentMode	Payment method/channel	name, currency, direction, limit, mode_type	ACH, IMPS, NEFT, SWIFT
:Workflow	Named process / flow in docs	name, system, trigger, steps_count, doc_id	CustomerOnboarding, OTPVerification
:Rule	Business/compliance rule	name, condition, value, unit, priority	TransactionLimit, RiskRatingThreshold
:Field	Data field / API parameter	name, type, mandatory, allowed_values, source	RiskRating, BeneficiaryAccount
:CustomerSegment	Customer category/type	name, region, tier, eligibility	RetailNRI, CorporateClient
:Concept	Domain concept not fitting above	name, domain, aliases	RiskRating, RemittanceInflow
:Chunk	Bridge: links graph → vector store	chunk_id, doc_id, section_path, vector_id	Points to Qdrant vector ID

Relationship Types (Edges)

RELATIONSHIP	FROM → TO	PROPERTIES	EXAMPLE QUERY IT ENABLES
[:CONTAINS]	Document → Section	order	"All sections in doc X"
[:HAS_CHUNK]	Section → Chunk	chunk_type	"Get text for section Y" → fetch from Qdrant

			Qdrant
[:MENTIONS]	Section → any entity	count, context	"Which docs mention IMPS?"
[:DEPENDS_ON]	System → System	criticality, failure_mode	"What does Genesys depend on?"
[:INTEGRATES_WITH]	System → System	protocol, direction	"How does System A talk to System B?"
[:NEXT_STEP]	Workflow → Workflow	condition, order	"Full flow from onboarding to transaction"
[:APPLIES_TO]	Rule → CustomerSegment / PaymentMode	scope	"Which rules apply to retail NRI customers?"
[:DEFINED_IN]	Any entity → Document + Section	version, date	"Where is ACH limit defined?"
[:CONFLICTS_WITH]	Entity → Entity	conflict_type, property	"Any conflicting ACH rules across docs?"
[:SUPERSEDES]	Document → Document	change_summary	"What changed between v1 and v2?"
[:TRIGGERS]	Event/Rule → Workflow	condition	"What triggers the OTP workflow?"
[:USES]	Workflow → PaymentMode / System / Field	mandatory	"Which workflows use IMPS?"
[:SAME_AS]	Entity → Entity	confidence	Deduplication: "ACH" = "Automated Clearing House"

Key design principle: Every graph entity must have a `:HAS_CHUNK` or `[:DEFINED_IN]` link back to a Section, which has a `:HAS_CHUNK` pointing to the Qdrant chunk_id. This is the bridge between graph traversal results and the vector store — after graph traversal finds relevant nodes, you fetch their associated text chunks from Qdrant/PostgreSQL to send to the LLM.

G3 // ENTITY EXTRACTION

Entity & Relationship Extraction Pipeline

This is the most complex new piece. It runs offline during ingestion and converts raw document text into structured graph nodes and edges.

> Entity extraction is a preprocessing step — it runs once per document (or when documents are updated) and is never part of the live query path. You use an LLM here specifically for structured extraction, not for answering. The output feeds Neo4j.

Extraction Architecture

Document Chunks (from Phase 3) → NLP Pre-filter (spaCy NER) → LLM Extractor (structured JSON) → Entity Resolver (deduplication) → Neo4j Loader (MERGE queries)

Step 1 — NLP Pre-filtering with spaCy

Step 1 — NER Pre-filtering with spaCy

Before sending chunks to the LLM (which is expensive), run spaCy to get a rough set of named entities. This also reduces LLM tokens by flagging which chunks are entity-dense vs pure boilerplate.

```
# Use spaCy for fast, cheap pre-filtering
import spacy
nlp = spacy.load("en_core_web_lg")

def is_entity_dense(chunk_text):
    doc = nlp(chunk_text)
    entities = [(ent.text, ent.label_) for ent in doc.ents]
    # If chunk has ≥3 named entities → send to LLM extractor
    # If chunk is boilerplate (version history, ToC) → skip
    return len(entities) >= 3, entities
```

Step 2 — LLM Structured Extraction (The Core Step)

For entity-dense chunks, send to a fast LLM (GPT-4o-mini or Claude Haiku) with a strict JSON extraction prompt. Use **Function Calling / Structured Output** — never free text — to guarantee parseable output.

```
SYSTEM PROMPT (Entity Extraction — Offline Preprocessing):
You are a knowledge graph entity extractor for banking documents.
Extract entities and relationships from the given text chunk.

Return ONLY valid JSON matching this exact schema:
{
  "entities": [
    {
      "id": "unique_snake_case_id",
      "label": "System|PaymentMode|Workflow|Rule|Field|CustomerSegment|Concept",
      "name": "canonical entity name",
      "properties": { "key": "value", ... },
      "source_text": "exact quote from chunk (≤20 words)"
    }
  ],
  "relationships": [
    {
      "from_id": "entity_id",
      "to_id": "entity_id",
      "type": "DEPENDS_ON|USES|APPLIES_TO|NEXT_STEP|CONFLICTS_WITH|...",
      "properties": { "key": "value" }
    }
  ]
}

Rules:
- Only extract entities explicitly stated in the text. Never infer.
- For ambiguous entity types, use "Concept".
- Use snake_case for all IDs.
- source_text must be a verbatim quote from the chunk.
- If no entities found, return {"entities": [], "relationships": []}


USER:
Chunk text: "The ACH payment mode is used for USD transactions up to $5000.
It integrates with the CoreBanking system via the PaymentGateway API.
Risk rating of HIGH automatically blocks ACH transactions."
```

```
RESPONSE (structured JSON):
{
  "entities": [
    { "id": "ach_payment", "label": "PaymentMode", "name": "ACH",
      "properties": { "currency": "USD", "limit": 5000 } },
    { "id": "core_banking", "label": "System", "name": "CoreBanking" },
    { "id": "risk_high", "label": "Rule", "name": "HighRiskACHBlock",
      "properties": { "condition": "risk_rating=HIGH", "effect": "block_ACH" } }
  ],
  "relationships": [
    { "from_id": "ach_payment", "to_id": "core_banking", "type": "INTEGRATES_WITH" },
    { "from_id": "risk_high", "to_id": "ach_payment", "type": "APPLIES_TO" }
  ]
}
```

Step 3 — Entity Resolution (Deduplication)

The Critical Deduplication Problem

Across 30 documents, "ACH", "Automated Clearing House", "ACH Payment", and "ACH Mode" all refer to the same entity. If stored as separate nodes, graph traversal breaks completely. You must resolve these before loading into Neo4j.

- **Exact match:** Normalize case, strip punctuation → merge identical names
- **Alias dictionary:** Maintain a banking domain alias map: ACH ↔ Automated Clearing House, IMPS ↔ Immediate Payment Service
- **Embedding similarity:** Embed entity names; if cosine similarity > 0.92 and same label → propose merge for human review
- **Neo4j MERGE:** Use `MERGE (n:PaymentMode {name: "ACH"})` — creates if not exists, updates if exists. This is the database-level deduplication guarantee

---

G4 // GRAPH POPULATION

Loading the Graph — Neo4j Population Strategy

How extracted entities are safely written into Neo4j with full traceability back to source chunks.

The MERGE Pattern — Idempotent Loading

Every write to Neo4j uses MERGE not CREATE. This means running the pipeline twice on the same document doesn't duplicate nodes — it updates them. Critical for when documents are versioned or re-ingested.

```
// Neo4j Cypher — Idempotent node creation

// 1. Create/update Document node
MERGE (d:Document {id: "ICICI_ORP_Genesys_FSD"})
SET d.name = "ICICI ORP Genesys Integration FSD",
    d.version = "1.2",
    d.system = "Genesys",
```

```
    d.ingested_at = datetime()

// 2. Create/update PaymentMode entity
MERGE (pm:PaymentMode {name: "ACH"})
SET pm.currency = "USD", pm.limit = 5000

// 3. Create/update relationship with source traceability
MERGE (pm)-[r:DEFINED_IN]->(d)
SET r.section = "1.1 Product Requirement",
    r.chunk_id = "doc_001_sec_1_1_chunk_03"  ← links back to Qdrant

// 4. Chunk bridge node (links graph → vector store)
MERGE (c:Chunk {chunk_id: "doc_001_sec_1_1_chunk_03"})
SET c.vector_id = "qdrant_vec_8821",  ← Qdrant point ID
    c.doc_id = "ICICI_ORP_Genesys_FSD"
MERGE (pm)-[:HAS_CHUNK]->(c)
```

Conflict Detection During Loading

Automatic Conflict Edge Creation

When loading a new entity, run a Cypher check: does this entity already exist with a different property value? If yes, auto-create a `:CONFLICTS_WITH` edge — so conflict queries work without any additional processing.

```
// After loading new ACH node from Doc_07 with limit=8000:
MATCH (a:PaymentMode {name: "ACH"})-[:DEFINED_IN]->(d1)
MATCH (b:PaymentMode {name: "ACH"})-[:DEFINED_IN]->(d2)
WHERE a.limit <> b.limit AND d1 <> d2
MERGE (a)-[:CONFLICTS_WITH {property: "limit", conflict_type: "value_mismatch"}]->(b)
```

G5 // QUERY ROUTING

Query Router — Vector vs Graph vs Hybrid

The router decides which retrieval path(s) to activate for each incoming user query. This is a classifier, not a heuristic.

Three Query Modes

Mode A: Vector Only

Factual, definitional, or procedural questions about a single concept

- "What is the purpose of the NYB Report?"
- "How does OTP verification work?"
- "What fields are in the customer profile?"

Mode B: Graph Only

Structural, relational, comparison, or dependency questions

- "Which systems does Genesys depend on?"
- "Are there any conflicting transaction limits?"
- "What triggers the OTP workflow?"

Mode C: Hybrid (Both)

Complex questions requiring both relationship context and full text

- "How do ACH limits differ between v1 and v2 and why?"
- "What is the complete onboarding-to-payment flow with all rules?"
- "Compare risk rating logic across all documents"

The Router Implementation

```
# Query router — runs before any retrieval
# Uses lightweight classifier (can be rule-based or small LLM call)

GRAPH_TRIGGER_SIGNALS = [
    # Comparison signals
    "differ", "compare", "difference", "vs", "versus", "between",
    # Relationship signals
    "depend", "integrat", "connect", "relat", "link",
    # Impact/chain signals
    "affect", "trigger", "cascade", "flow", "chain", "impact",
    # Conflict/consistency signals
    "conflict", "contradict", "inconsist", "mismatch",
    # Aggregation signals
    "all document", "across", "every", "which doc", "how many",
]

def route_query(query: str) -> str:
    q_lower = query.lower()
    needs_graph = any(sig in q_lower for sig in GRAPH_TRIGGER_SIGNALS)
    needs_vector = True  # Always retrieve text too

    if needs_graph and needs_vector:
        return "HYBRID"   # Run both paths
    elif needs_graph:
        return "GRAPH"    # Graph only (rare)
    else:
        return "VECTOR"   # Original pipeline
```

Practical note: In practice, almost all non-trivial queries benefit from HYBRID mode. Default to HYBRID for any query longer than 8 words. Pure VECTOR mode is for simple factual lookups. Pure GRAPH mode is rarely used alone since the LLM still needs text context to formulate an answer.

G6 // GRAPH RETRIEVAL

Cypher Query Patterns for Banking Queries

The system auto-generates Cypher queries from user questions. Here are the core patterns covering every query type your banking team will ask.

Pattern 1 — Dependency / Impact Query

```
// User: "What does the Genesys system depend on and what happens if any fail?"
// Auto-generated Cypher:

MATCH (s:System {name: "Genesys"})-[r:DEPENDS_ON]->(dep)
OPTIONAL MATCH (s)-[r2:INTEGRATES_WITH]->(dep2)
OPTIONAL MATCH (dep)-[:HAS_CHUNK]->(chunk:Chunk)
RETURN
    dep.name AS dependency,
    r.failure_mode AS failure_mode,
    r.criticality AS criticality,
    chunk.chunk_id AS source_chunk  ← fetch this from Qdrant for LLM context
```

Pattern 2 — Cross-Document Comparison

```
// User: "How do ACH transaction limits differ across all documents?"

MATCH (pm:PaymentMode {name: "ACH"})-[r:DEFINED_IN]->(d:Document)
RETURN
    d.name AS document,
    d.version AS version,
    pm.limit AS limit,
    pm.currency AS currency,
    r.section AS defined_in_section,
    r.chunk_id AS chunk_id
ORDER BY d.version
```

Pattern 3 — Conflict Detection

```
// User: "Are there any conflicting rules about transaction limits across documents?"

MATCH (a)-[c:CONFLICTS_WITH]->(b)
MATCH (a)-[:DEFINED_IN]->(d1:Document)
MATCH (b)-[:DEFINED_IN]->(d2:Document)
RETURN
    labels(a)[0] AS entity_type,
    a.name AS entity_a,
    d1.name AS in_document_1,
    b.name AS entity_b,
    d2.name AS in_document_2,
    c.property AS conflicting_property,
    c.conflict_type AS conflict_type
```

Pattern 4 — Full Process Chain Traversal

```
// User: "What is the complete workflow from customer registration to first remittance?"

MATCH path = (start:Workflow {name: "CustomerOnboarding"})
              -[:NEXT_STEP*1..10]->
              (end:Workflow)
WITH nodes(path) AS steps
UNWIND steps AS step
OPTIONAL MATCH (step)-[:HAS_CHUNK]->(chunk:Chunk)
RETURN
    step.name AS workflow_step,
    step.trigger AS trigger_condition,
    step.system AS handled_by_system,
    chunk.chunk_id AS detail_chunk
ORDER BY index ← preserves step sequence
```

Pattern 5 — Risk Rule Analysis

```
// User: "What rules apply to HIGH risk rating customers and which payment modes are blocked?"

MATCH (rule:Rule)
WHERE rule.condition CONTAINS "HIGH"
MATCH (rule)-[:APPLIES_TO]->(target)
OPTIONAL MATCH (rule)-[:DEFINED_IN]->(d:Document)
RETURN
    rule.name AS rule_name,
    rule.condition AS condition,
    rule.effect AS effect,
    labels(target)[0] AS applies_to_type,
    target.name AS applies_to,
    d.name AS source_document
```

Auto-Generating Cypher from Natural Language

Cypher Generation Prompt (LLM at Query Time)

When the router decides GRAPH or HYBRID mode, an LLM generates the Cypher query based on the graph schema. This is a separate, fast LLM call that happens before retrieval.

```
SYSTEM: You are a Cypher query generator for a Neo4j banking knowledge graph.
Schema: [Document, Section, System, PaymentMode, Workflow, Rule, Field, Chunk]
Relationships: [CONTAINS, HAS_CHUNK, DEPENDS_ON, INTEGRATES_WITH, NEXT_STEP,
                APPLIES_TO, DEFINED_IN, CONFLICTS_WITH, USES, MENTIONS, SAME_AS]

Rules:
- Always include HAS_CHUNK or DEFINED_IN to return chunk_ids for text retrieval
- Use OPTIONAL MATCH for relationships that may not exist
- Never use LIMIT less than 20 (we need complete results)
- Return chunk_ids alongside graph data so text can be fetched

USER: How does risk rating affect which payment modes a customer can use?
CYPHER:
```

G7 // CLARIFICATION

GraphQL vs Cypher — Which One and Why

These are often confused. They serve completely different roles in this system.

GRAPHQL — NOT WHAT YOU NEED INTERNALLY

- A query language for **APIs between frontend and backend**
- Designed for clients to request exactly the fields they want over HTTP
- Has no concept of graph traversal or path following
- Used by companies like Facebook to serve data to web/mobile apps
- Would be used here only if you build a **public API** that external apps query

CYPHER — WHAT YOU USE TO QUERY NEO4J ✓

- Native query language of Neo4j, designed specifically for graphs
- Supports pattern matching across nodes and edges: `MATCH (a)-[:DEPENDS_ON*1..5]->(b)`
- Built for traversal, path finding, aggregation, and relationship filtering
- Runs server-side inside Neo4j — your Python backend calls it via the neo4j driver
- This is what your Query Engine generates and executes

Summary: Your Python backend uses the `neo4j` driver to run Cypher queries directly against Neo4j. GraphQL is irrelevant to the internal graph retrieval. You would only introduce GraphQL if your system later exposes a query API to an external client — even then, it would sit in front of Cypher, not replace it.

Optional: Neo4j GraphQL API (Future Enhancement)

Neo4j does offer a **GraphQL library** that auto-generates a GraphQL API from your schema. This would only make sense if external teams or front-end apps need to query the knowledge graph directly — not for the RAG pipeline itself, which always goes through Python.

G8 // RESULT FUSION

Merging Vector + Graph Results Before the LLM

Both retrieval paths produce different output formats. They must be merged, deduplicated, and structured before the reranker and LLM receive them.

What Each Path Returns

Vector Path Returns

- Top-8 text chunks (after reranking)
- Each chunk: raw text + breadcrumb + source doc
- Format: unstructured prose, tables, workflow descriptions
- Good for: detailed explanation, nuanced context

Graph Path Returns

- Structured rows: entity names, properties, relationships
- Each row includes chunk_ids for referenced entities
- Format: tabular / structured data (JSON rows)
- Good for: relationship maps, comparisons, conflict tables

Fusion Strategy

```
def fuse_results(vector_chunks, graph_rows):
    context_blocks = []

    # 1. Convert graph rows → structured context string
    if graph_rows:
        graph_summary = format_graph_as_table(graph_rows)
        context_blocks.append({
            "type": "graph_facts",
            "content": graph_summary,
            "label": "[RELATIONSHIP DATA from Knowledge Graph]"
        })

    # 2. Fetch text chunks linked from graph chunk_ids
    graph_chunk_ids = [row["chunk_id"] for row in graph_rows if row.get("chunk_id")]
    graph_text_chunks = fetch_from_qdrant(graph_chunk_ids)

    # 3. Merge and deduplicate all text chunks
    all_chunks = vector_chunks + graph_text_chunks
    seen_ids = set()
    unique_chunks = []
    for c in all_chunks:
        if c["chunk_id"] not in seen_ids:
            unique_chunks.append(c)
            seen_ids.add(c["chunk_id"])

    # 4. Rerank all text chunks together (same reranker as before)
    reranked = cross_encoder_rerank(query, unique_chunks, top_k=8)

    # 5. Final context: graph facts (structured) + reranked text chunks
    context_blocks += [{"type": "text", "content": c} for c in reranked]
    return context_blocks
```

LLM Prompt Structure for Hybrid Context

```
SYSTEM: [Grounding rules — same as before, never use training knowledge]

[GRAPH RELATIONSHIP DATA]
```

```
The following structured facts were retrieved from the knowledge graph:

| Entity       | Relationship    | Target          | Source Document      |
|-------------|----------------|----------------|---------------------|
| ACH (5000)  | CONFLICTS_WITH  | ACH (8000)      | FSD_v1 vs FSD_v2    |
| Genesys     | DEPENDS_ON      | RiskEngine      | Genesys_Integration |
| HighRiskRule| APPLIES_TO      | ACH PaymentMode | ICICI_ORP_FSD       |

[TEXT CONTEXT FROM DOCUMENTS]
--- Doc: ICICI_ORP_FSD | Section: Payment Processing ---
{chunk_1_text}

--- Doc: ICICI_ORP_FSD_v2 | Section: Payment Limits ---
{chunk_2_text}

[USER QUESTION]
{query}

INSTRUCTIONS:
- Use BOTH the graph relationship data AND the text context to answer
- The graph data gives you structural facts; the text gives you detail and nuance
- Cite specific sources for every claim
- For conflicts found in graph data, explicitly flag them in your answer
```

G9 // FULL ARCHITECTURE

Complete Updated System Architecture

Everything together — original vector RAG pipeline + full graph layer. Purple = new additions.

```
╔═══════════════════════ INGESTION PIPELINE (Offline — runs once per doc) ═══════════════════╗
║                                                                                            ║
║  [.docx / .pdf Files]                                                                      ║
║       │                                                                                    ║
║       ▼                                                                                    ║
║  [python-docx Parser] ──────────────► paragraphs + heading structure + tables + metadata   ║
║       │                                                                                    ║
║       ├──────────────────── EXISTING PATH ──────────────────────────────────────────────── ║
║       │                                                                                    ║
║       ▼                                                                                    ║
║  [Hierarchical Chunker] ─► Parent + Child nodes with breadcrumb paths                       ║
║       │                                                                                    ║
║       ├─ child chunks ─► [Embedding Model] ──────────────────► [Qdrant Vector DB]           ║
║       ├─ child chunks ─► [BM25 Indexer] ──────────────────────► [BM25 Index]               ║
║       └─ parent chunks──────────────────────────────────────► [PostgreSQL Doc Store]       ║
║                                                                                            ║
║       │                                                                                    ║
║       ├──────────────────── NEW GRAPH PATH ──────────────────────────────────────────────── ║
║       │                                                                                    ║
```

```
│       ▼                                                                                                  │
│  [spaCy NER Pre-filter] ──► flag entity-dense chunks only                                                │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [LLM Entity Extractor] ──► structured JSON: {entities[], relationships[]}                               │
│       │                     (runs offline — GPT-4o-mini or Claude Haiku)                                 │
│       ▼                                                                                                  │
│  [Entity Resolver] ────────► deduplicate + alias match + normalize names                                 │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Conflict Detector] ──────► auto-create :CONFLICTS_WITH edges on property mismatches                    │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Neo4j Loader] ────────────► MERGE nodes + relationships + Chunk bridges → [Neo4j Graph DB]             │
│                                                                                                          │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────── QUERY PIPELINE (Runtime — per user query) ────────────────────────────┐
│                                                                                                          │
│  [User Query]                                                                                            │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Query Processor]: classify intent → expand synonyms → generate sub-queries → extract filters           │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Query Router] ──────────────────────────────────────────────────────────────────────────────────────   │
│       │                       │                                   │                                      │
│       ▼ VECTOR / HYBRID       ▼ GRAPH / HYBRID                    │                                      │
│       │                       │                                   │                                      │
│  [Qdrant] (dense-40)     [Cypher Generator LLM]                   │                                      │
│  [BM25]  (sparse-20)          │                                   │                                      │
│       │                       ▼                                   │                                      │
│       │               [Neo4j Cypher Query] ──► structured rows + chunk_ids                               │
│       │                       │                                                                          │
│       ▼                       ▼                                                                          │
│  [RRF Fusion] ◄──────── [Graph Chunk Fetch] ──► fetch linked text from Qdrant/PostgreSQL                 │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Parent Node Fetcher] (PostgreSQL) → upgrade child chunks to richer parent chunks                       │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Cross-Encoder Reranker] (no LLM) → top-8 scored chunks, confidence threshold 0.5                       │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Context Assembler] → merge: graph structured facts + reranked text chunks                              │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Grounded LLM Prompt] → system: context-only rules + graph facts + text chunks → answer + citations    │
│       │                                                                                                  │
│       ▼                                                                                                  │
│  [Faithfulness Validator] → verify answer claims against retrieved context                               │
│       │                                                                                                  │
```

```
▼
[Response]: answer + cited sections + graph relationships shown + confidence score
```

Updated Storage Layer Summary

STORE	WHAT IT HOLDS	QUERIED BY	STATUS
Qdrant	Dense vector embeddings of child chunks	Semantic similarity search	EXISTING
BM25 Index	Inverted index of child chunk tokens	Keyword / acronym search	EXISTING
PostgreSQL	Full parent + child chunks + metadata	Parent node fetcher	EXISTING
Neo4j	Knowledge graph: entities + relationships + Chunk bridge nodes	Cypher traversal queries	NEW

G10 // ROADMAP UPDATE

Updated Implementation Roadmap

Original Phases 1–10 remain unchanged. Graph work begins after the vector pipeline is stable and tested.

SPRINT	MILESTONE	KEY WORK	NOTE
Weeks 1–8	**Original RAG Phases 1–10**	Full vector pipeline: parsing → chunking → embedding → reranker → LLM	EXISTING PLAN
Week 9	**Graph Schema + Neo4j Setup**	Install Neo4j, define schema, write MERGE templates, set up Chunk bridge	NEW
Week 10	**Entity Extraction Pipeline**	spaCy filter, LLM extractor prompt, structured JSON output, entity resolver	NEW
Week 11	**Graph Population**	Run extractor on all 30 docs → load into Neo4j → verify graph integrity → conflict auto-detection	NEW
Week 12	**Query Router + Cypher Generator**	Signal-based router, Cypher generation prompt, test all 6 query patterns	NEW
Week 13	**Result Fusion + Context Assembler**	Merge graph rows + vector chunks, dedup, combined reranker pass, updated LLM prompt	NEW
Week 14	**End-to-End Testing + Eval**	Add 30 relationship/comparison queries to golden set, test all modes, tune router thresholds	NEW
Week 15	**Production Hardening**	Neo4j clustering, graph update on new doc ingestion, monitoring, full team rollout	NEW

⚡ Graph Quality = Entity Extraction Quality

The single biggest risk in Graph RAG is entity extraction quality. If the LLM extractor misnames entities, misses relationships, or incorrectly resolves aliases, the graph becomes untrustworthy. Allocate significant effort to Week 10–11: manually review extracted entities from your first 5 documents before mass-processing all 30. Build a small entity correction UI so your banking team can flag wrong extractions. The graph is only as accurate as what went into it.

📌 Final Recommendation on Architecture Choice

- **Vector RAG alone:** Handles ~70% of queries well. Fast to build, easy to maintain.
- **Graph RAG added:** Handles the remaining ~30% that involve relationships, comparisons, and cross-document reasoning. This is the right call for a banking team with rich, interconnected FSDs.
- **Build order matters:** Vector first, graph second. A working vector system is the foundation — graph is the intelligence layer on top.